TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Algoritmos y Estructuras de Datos I

Escuela de Ingeniería en Computadores

Informe Tarea 2:

Benchmark de algoritmos de ordenamiento

Profesor: Luis Alonso Barboza Artavia

Estudiantes:

Ignacio José Apuy Anchia

Ulfran Ricardo Rodriguez Cordero

II Semestre 2026

Versión: 1.0

TEC | Tecnológico de Costa Rica

Parte I: Predicciones.....4
Parte II: Implementaciones: Prompt de IA utilizado.....6
A. Sistema de medición y almacenamiento de resultados.....7
B. Uso de inteligencia artificial.....7
C. Validación de las implementaciones [Array.Sort()].....8
Parte III: Experimentos y Tipos de Entradas: Prompt 3.....9
A. Flujo del programa de pruebas.....9
B. Flujo del programa de procesamiento.....9
C. Plan de benchmark.....10
D. Ejecución.....11
Parte IV: Análisis de Resultados:.....12
A. Tablas de tiempos, en milisegundos.....12
A.1. Selection, Bubble, Insertion y Quick.....12
A.2. Shell, Merge y Radix, n de 1 000 a 32 000.....14
A.3. Shell, Merge y Radix, n de 64 000 a 1 024 000.....15
B. Tabla de tiempos, en milisegundos, prueba especial de Radix (n = 100000).....16
C. Tabla de razones t(2n)/t(n) (Cercano a 2 es lineal, un poco más de 2 es n log n y cercano a 4 es cuadrático).....17
C.1. Selection, Bubble, Insertion y Quick.....17
C.2. Shell, Merge y Radix.....19
D. Tabla de comparación de algoritmos.....20
D.1. Tabla de comparación entre Insertion y Quick según entrada.....20
D.2. Tabla de comparación entre Insertion y Quick según entrada.....20
E. Comparaciones entre la tabla de predicciones y tabla de resultados.....21
Referencias y Declaraciones de Uso de IA:.....23
Ulfran Rodriguez.....23
Prompt 1: Análisis del enunciado.....23
Prompt 2: Algoritmos y array:.....30
Ignacio Apuy.....31
Prompt 3: Programa de pruebas:.....31
Prompt 4: Programa graficador.....32
Prompt 5 y 6: Arreglos mínimos de errores.....33

TEC | Tecnológico de Costa Rica

Parte I: Predicciones

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Justificacion
Selection	O(n^2)	O(n^2)	O(n^2)	El algoritmo se basa en dos estructuras de iteración que recorren el n cantidad de elementos en el arreglo 2 veces
Bubble	O(n^2)	O(n^2)	O(n^2)	El algoritmo se basa en dos estructuras de iteración que recorren el n cantidad de elementos en el arreglo 2 veces
Insertion	O(n)	O(n^2)	O(n^2)	En este caso si el arreglo es uno sencillo el recorrido de la lista solo involucra un swap sin necesidad de recorrer la lista otra vez
Shell	Cuando el n es solo 2	Depende del incremento del n	Depende del incremento del n	Shell Sort mejora Insertion Sort al comparar elementos que se encuentran separados por determinadas distancias. Sim embargo

TEC | Tecnológico de Costa Rica

Merge	O (n log n)	O (n log n)	O (n log n)	Merge Sort utiliza la estrategia divide y vencerás. Divide el arreglo en partes más pequeñas, ordena cada parte y posteriormente combina los resultados.
QuickSort	O (n log n)	O (n log n)	O(n^2)	Este algoritmo se compara por agarrar un pivote que divide a los mayores a su derecha y menores a la izquierda, incluso si al mismo pivote le toca cambiar, de esta manera llegar a hacerlo eficiente, sin embargo si se tiene que llegar a hacer una doble iteración empieza a hacer el ordenamiento tardado
Radix	O (n log n)	O (n log n)	O (n log n)	Ordena los números procesando sus dígitos, desde el menos significativo hasta el más significativo, rendimiento depende de la cantidad de elementos y de la cantidad de dígitos que debe procesar.

TEC | Tecnológico de Costa Rica

Parte II: Implementaciones [Prompt 1: Análisis del enunciado]

Para el desarrollo de esta parte de la tarea se implementaron los siete algoritmos de ordenamiento solicitados utilizando el lenguaje C#, mediante un proyecto de consola.

La implementación se organizó en diferentes módulos para facilitar su mantenimiento y comprensión. Los algoritmos se encuentran separados en archivos individuales dentro de la carpeta Algoritmos, lo que permite ejecutar cada uno de manera independiente.

Los algoritmos implementados fueron:

- Selection Sort.
- Bubble Sort.
- Insertion Sort.
- Shell Sort.
- Merge Sort.
- QuickSort.
- Radix Sort.

Para QuickSort se seleccionó como pivote el último elemento del arreglo. En Shellsort se utilizó una secuencia de brechas que se reduce a la mitad en cada iteración, mientras que Radix Sort se implementó mediante el método LSD (Least Significant Digit), utilizando base 10.

Adicionalmente, se desarrolló un generador de arreglos encargado de producir los cuatro tipos de entrada requeridos: aleatorios, ascendentes, descendentes y con valores duplicados entre 0 y 10.

TEC | Tecnológico de Costa Rica

A. Sistema de medición y almacenamiento de resultados

Se implementó un módulo de benchmarking utilizando la clase *Stopwatch* de C#. Este módulo permite medir el tiempo de ejecución de cada algoritmo para diferentes tamaños de arreglos y tipos de entrada.

La generación de los datos se realiza antes de iniciar el cronómetro, de manera que el tiempo registrado corresponde únicamente a la ejecución del algoritmo de ordenamiento.

Los resultados se almacenan en archivos CSV, incluyendo información sobre el tamaño del arreglo, el tipo de entrada, el algoritmo utilizado y el tiempo de ejecución.

Posteriormente, se desarrolló un script en Python que permite procesar los archivos CSV, calcular las medianas de las mediciones y generar las gráficas utilizadas en el análisis experimental.

B. Uso de inteligencia artificial

Se utilizó inteligencia artificial como herramienta de apoyo durante el desarrollo del proyecto, principalmente para orientar la implementación de los algoritmos, estructurar el sistema de medición experimental y facilitar el procesamiento de los resultados.

Los principales objetivos de las consultas realizadas fueron:

1. Obtener orientación para implementar los algoritmos de ordenamiento en C#.
2. Desarrollar un generador de arreglos para los diferentes tipos de entrada.
3. Implementar un sistema de medición de tiempos utilizando Stopwatch.
4. Procesar los resultados experimentales mediante Python y generar las gráficas correspondientes.

Los prompts utilizados y las correcciones realizadas durante el desarrollo se documentan como parte de la evidencia del uso de inteligencia artificial.

TEC | Tecnológico de Costa Rica

C. Validación de las implementaciones [Prompt 2: Algoritmos y array:]

```csharp
using TareaExtraclase2.Algoritmos;

int[] original = { 64, 25, 12, 22, 11, 90, 3, 3, 0, 500000 };

void Probar(string nombre, Action<int[]> algoritmo)
{
    int[] copia = (int[])original.Clone();   // copia para no tocar el original
    algoritmo(copia);
    Console.WriteLine($"{nombre,-10}: {string.Join(", ", copia)}");
}

Probar("Selection", SelectionSort.Ordenar);
Probar("Bubble", BubbleSort.Ordenar);
Probar("Insertion", InsertionSort.Ordenar);
Probar("Shell", ShellSort.Ordenar);
Probar("Merge", MergeSort.Ordenar);
Probar("Quicksort", QuickSort.Ordenar);
Probar("Radix", RadixSort.Ordenar);
```

TEC | Tecnológico de Costa Rica

Parte III: Experimentos y Tipos de Entradas

A. Flujo del programa de pruebas [Prompt 3: Programa de pruebas:]

1. Se le solicita al usuario escoger un algoritmo para probar, junto a un minN y maxN que definirán el inicio y fin de los incrementos.
2. El programa genera el primer tipo de array desde n=minN hasta n=minN*2.
3. Se inicia el contador y comienza el ordenamiento.
4. Al finalizar se detiene el contador y se guarda el resultado en el csv.
5. Se repite el proceso con el siguiente tipo de array (Ordenado ascendente), igual con los 2 restantes.
6. Una vez ordenados los 4 tipos, se procede a duplicar el n actual.
7. El programa se repite hasta alcanzar maxN.

B. Flujo del programa de procesamiento [Prompt 4: Programa graficador]

1. Se obtienen los csv en la carpeta Resultados
2. Se exporta un csv con los datos crudos.
3. Se obtiene la mediana de los datos en un solo archivo.
4. Se genera un csv con las razones calculadas T(2n)/T(n)
5. Se generan dos gráficas log-log, una con todos los algoritmos, y otra con quicksort e insertion según su entrada.

TEC | Tecnológico de Costa Rica

C. Plan de benchmark

Algoritmo	minN	maxN	Tamaños esperados a guardar	Repeticiones
Bubble Sort	1,000	16,000	1000, 2000, 4000, 8000, 16000	5
Insertion Sort	1,000	16,000	1000, 2000, 4000, 8000, 16000	5
Merge Sort	1,000	1,024,000	1000, 2000, 4000, …, 1024000	5
Quick Sort	1,000	1,024,000	1000, 2000, 4000, …, 32000 Esto debido a que pruebas anteriores provocan un desbordamiento de pila en n=32000.	5
Radix Sort	1,000	1,024,000	1000, 2000, 4000, …, 1024000	5
Selection Sort	1,000	16,000	1000, 2000, 4000, 8000, 16000	5
Shell Sort	1,000	1,024,000	1000, 2000, 4000, …, 1024000	5
Radix Sort Especial	n = 10000		El programa ya fija n=10000, sólo cambia dígitos. 10000, 3 dígitos. 10000, 6 dígitos. 10000, 9 dígitos.	5

TEC | Tecnológico de Costa Rica

D. Ejecución.

Las pruebas se realizan con el menor número posible de procesos abiertos para intentar conseguir los mejores resultados.

Una vez realizadas las pruebas, se ejecuta el programa de procesamiento y los datos están listos para la tabulación.

TEC | Tecnológico de Costa Rica

Parte IV: Análisis de Resultados:

A. Tablas de tiempos, en milisegundos.

A.1. Selection, Bubble, Insertion y Quick.

Algoritmo	Entrada	1000	2000	4000	8000	16000	32000
Selection	Aleatorio	1.4827	6.2818	25.21	96.44	383.73	N/A
Selection	Ascendente	1.4147	5.7485	24.96	95.75	383.66	N/A
Selection	Descendente	1.4929	6.0211	25.11	100.41	403.46	N/A
Selection	Duplicados (0-10)	1.4739	5.7735	23.57	94.91	382.24	N/A
Bubble	Aleatorio	3.6517	16.66	63.92	259.12	1021.4	N/A
Bubble	Ascendente	1.5658	6.2041	25.73	105.47	420.26	N/A
Bubble	Descendente	2.9692	12.53	51.01	200.41	806.16	N/A
Bubble	Duplicados (0-10)	3.7643	16.98	62.57	250.07	1000.4	N/A
Insertion	Aleatorio	0.9269	3.6539	16.18	62.26	243.45	N/A
Insertion	Ascendente	0.0045	0.009	0.0181	0.0361	0.0721	N/A
Insertion	Descendente	1.9441	7.2	36.03	121.51	485.06	N/A
Insertion	Duplicados (0-10)	0.8393	3.3067	15.26	56.39	220.98	N/A

TEC | Tecnológico de Costa Rica

Quick	Aleatorio	0.1038	0.2225	0.5571	1.1005	2.2661	4.9429
Quick	Ascendente	3.5721	12.35	49.02	255.60	771.42	Overflow
Quick	Descendente	2.1751	8.8652	41.91	148.16	554.75	Overflow
Quick	Duplicados (0-10)	0.3141	1.1618	14.74	18.29	70.09	Overflow

TEC | Tecnológico de Costa Rica

A.2. Shell, Merge y Radix, n de 1 000 a 32 000

Algoritmo	Entrada	1000	2000	4000	8000	16000	32000
Shell	Aleatorio	0.1596	0.3548	0.8265	1.8873	4.3313	10.46
Shell	Ascendente	0.0377	0.0845	0.1903	0.4349	0.9525	2.0664
Shell	Descendente	0.0681	0.1515	0.3299	0.7225	1.5862	3.4121
Shell	Duplicados (0-10)	0.0982	0.2169	0.465	1.0073	2.2911	4.9699
Merge	Aleatorio	0.1536	0.3316	0.716	1.6511	3.3048	7.1084
Merge	Ascendente	0.0977	0.2058	0.5494	0.9511	2.0412	4.72
Merge	Descendente	0.1021	0.2129	0.4635	0.9771	2.0969	4.5189
Merge	Duplicados (0-10)	0.1356	0.2814	0.5958	1.2808	2.6764	6.8101
Radix	Aleatorio	0.1809	0.3537	0.7296	1.4199	2.8786	5.8061
Radix	Ascendente	0.1973	0.3561	0.7209	1.4382	2.9047	5.7908
Radix	Descendente	0.1786	0.3555	0.7669	1.4321	2.9195	6.3456
Radix	Duplicados (0-10)	0.0544	0.1116	0.2179	0.4397	0.884	1.8112

TEC | Tecnológico de Costa Rica

A.3. Shell, Merge y Radix, n de 64 000 a 1 024 000

Algoritmo	Entrada	64000	128000	256000	512000	1024000
Shell	Aleatorio	26.67	58.43	131.53	315.55	759.27
Shell	Ascendente	4.5103	9.9806	21.34	45.12	96.73
Shell	Descendente	8.2796	15.90	33.63	70.92	149.48
Shell	Duplicados (0-10)	10.66	22.41	52.84	101.23	213.12
Merge	Aleatorio	17.66	33.94	71.17	147.46	310.77
Merge	Ascendente	10.78	20.10	42.08	88.39	186.32
Merge	Descendente	10.38	21.61	43.87	92.77	197.50
Merge	Duplicados (0-10)	12.37	26.65	60.87	113.49	239.04
Radix	Aleatorio	14.80	24.64	49.77	103.91	199.23
Radix	Ascendente	13.61	24.49	49.10	100.99	203.14
Radix	Descendente	12.89	25.05	49.66	97.97	199.18
Radix	Duplicados (0-10)	3.7523	7.5961	15.48	31.14	65.45

TEC | Tecnológico de Costa Rica

B. Tabla de tiempos, en milisegundos, prueba especial de Radix (n = 100000)

Dígitos (d)	Valores	Tiempo (ms)	t(d)/t(3)	Teórico d/3	ms por dígito
3	0 – 999	8.9461	1.00	1.00	2.98
6	0 – 999 999	18.9399	2.12	2.00	3.16
9	0 – 999 999 999	32.4867	3.63	3.00	3.61

TEC | Tecnológico de Costa Rica

C. Tabla de razones t(2n)/t(n) (Cercano a 2 es lineal, un poco más de 2 es n log n y cercano a 4 es cuadrático)

C.1. Selection, Bubble, Insertion y Quick

Algoritmo	Entrada	2000	4000	8000	16000	32000	Prom.	Mediana
Selection	Aleatorio	4.24	4.01	3.83	3.98	N/A	4.01	4.00
Selection	Ascendente	4.06	4.34	3.84	4.01	N/A	4.06	4.04
Selection	Descendente	4.03	4.17	4.00	4.02	N/A	4.06	4.03
Selection	Duplicados (0-10)	3.92	4.08	4.03	4.03	N/A	4.01	4.03
Bubble	Aleatorio	4.56	3.84	4.05	3.94	N/A	4.10	4.00
Bubble	Ascendente	3.96	4.15	4.10	3.98	N/A	4.05	4.04
Bubble	Descendente	4.22	4.07	3.93	4.02	N/A	4.06	4.05
Bubble	Duplicados (0-10)	4.51	3.69	4.00	4.00	N/A	4.05	4.00
Insertion	Aleatorio	3.94	4.43	3.85	3.91	N/A	4.03	3.93
Insertion	Ascendente	2.00	2.01	1.99	2.00	N/A	2.00	2.00
Insertion	Descendente	3.70	5.00	3.37	3.99	N/A	4.02	3.85

TEC | Tecnológico de Costa Rica

Insertion	Duplicados (0-10)	3.94	4.61	3.70	3.92	N/A	4.04	3.93
Quick	Aleatorio	2.14	2.50	1.98	2.06	2.18	2.17	2.14
Quick	Ascendente	3.46	3.97	5.21	3.02	Overflow	3.91	3.71
Quick	Descendente	4.08	4.73	3.54	3.74	Overflow	4.02	3.91
Quick	Duplicados (0-10)	3.70	12.68	1.24	3.83	Overflow	5.36	3.77

TEC | Tecnológico de Costa Rica

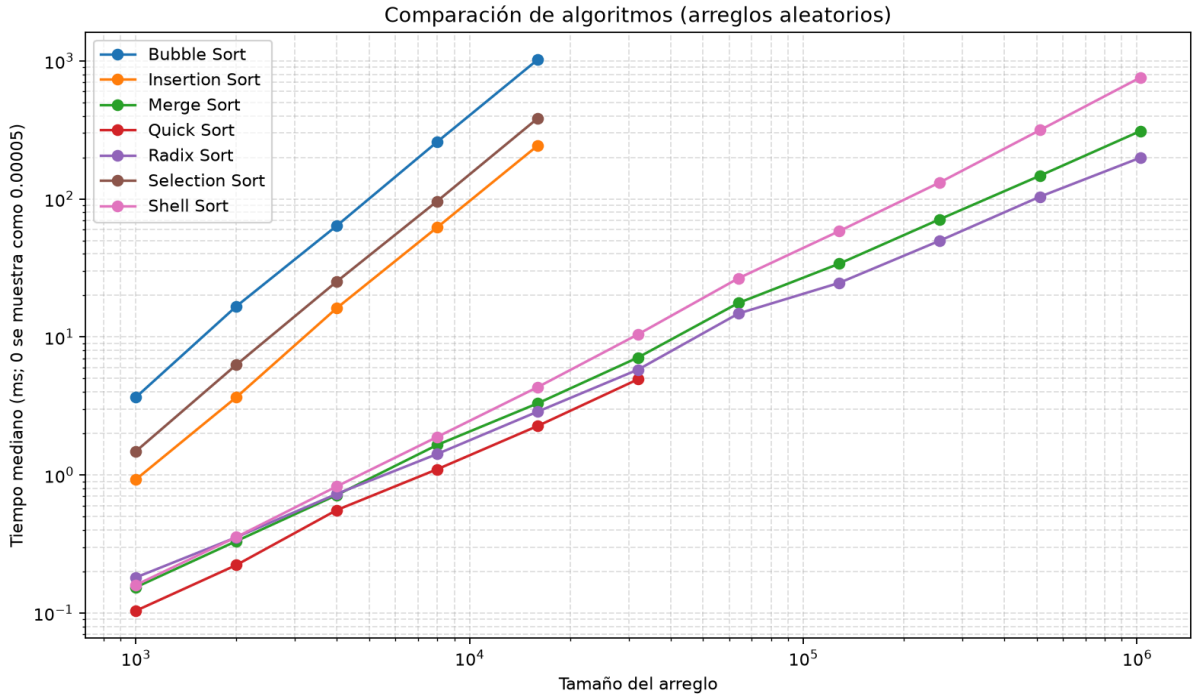
C.2. Shell, Merge y Radix

Algoritmo	Entrada	2000	8000	16000	64000	128000	512000	1024000	Prom.	Mediana
Shell	Aleatorio	2.22	2.28	2.29	2.55	2.19	2.40	2.41	2.33	2.31
Shell	Ascendente	2.24	2.29	2.19	2.18	2.21	2.11	2.14	2.19	2.19
Shell	Descendente	2.22	2.19	2.20	2.43	1.92	2.11	2.11	2.16	2.16
Shell	Duplicados (0-10)	2.21	2.17	2.27	2.14	2.10	1.92	2.11	2.16	2.16
Merge	Aleatorio	2.16	2.31	2.00	2.49	1.92	2.07	2.11	2.15	2.13
Merge	Ascendente	2.11	1.73	2.15	2.28	1.87	2.10	2.11	2.14	2.11
Merge	Descendente	2.09	2.11	2.15	2.30	2.08	2.11	2.13	2.13	2.12
Merge	Duplicados (0-10)	2.08	2.15	2.09	1.82	2.15	1.86	2.11	2.12	2.11
Radix	Aleatorio	1.96	1.95	2.03	2.55	1.66	2.09	1.92	2.02	2.02
Radix	Ascendente	1.80	2.00	2.02	2.35	1.80	2.06	2.01	2.01	2.01
Radix	Descendente	1.99	1.87	2.04	2.03	1.94	1.97	2.03	2.02	2.01
Radix	Duplicados (0-10)	2.05	2.02	2.01	2.07	2.02	2.01			

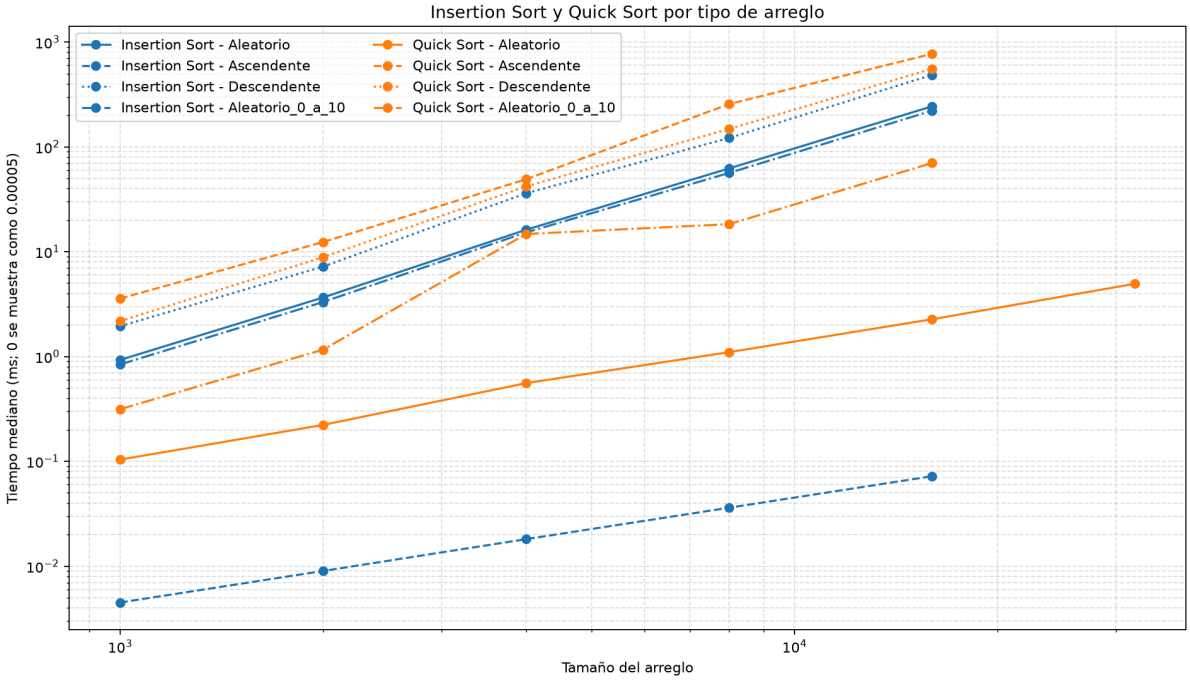
TEC | Tecnológico de Costa Rica

D. Tabla de comparación de algoritmos.

D.1. Tabla de comparación entre Insertion y Quick según entrada.



D.2. Tabla de comparación entre Insertion y Quick según entrada.



TEC | Tecnológico de Costa Rica

E. Comparaciones entre la tabla de predicciones y tabla de resultados

Selection Sort: En la Parte I se predijo un comportamiento (O(n^2)) para todos los casos. Los resultados experimentales respaldan esta predicción, ya que las medianas de las razones (t(2n)/t(n)) fueron 4.00 para entradas aleatorias, 4.04 para ascendentes, 4.03 para descendentes y 4.03 para duplicados. Esto demuestra que el tiempo de ejecución crece aproximadamente cuatro veces al duplicar el tamaño del arreglo, independientemente del tipo de entrada.

Bubble Sort: Se predijo una complejidad (O(n^2)) para todos los casos, lo cual coincide con los resultados obtenidos. Las medianas de las razones fueron 4.00 para entradas aleatorias, 4.04 para ascendentes, 4.05 para descendentes y 4.00 para duplicados. Por lo tanto, se confirma el crecimiento cuadrático esperado, incluso cuando los datos ya se encuentran ordenados, debido a que la implementación realiza todas las pasadas sin detenerse anticipadamente.

Insertion Sort: En la Parte I se predijo una complejidad (O(n)) para el mejor caso y (O(n^2)) para el promedio y peor caso. Las mediciones confirman esta diferencia: para entradas ascendentes se obtuvo una mediana de 2.00, correspondiente a un crecimiento lineal, mientras que para entradas aleatorias, descendentes y con duplicados se obtuvieron 3.93, 3.85 y 3.93, respectivamente, valores cercanos a 4 que respaldan el comportamiento cuadrático.

Shell Sort: En la parte 1 se hizo una predicción de complejidad de O(n) para su mejor caso y al peor caso se le asignó la predicción de O(n^2) junto al promedio. Las mediciones confirman esta diferencia: en el caso de las entradas ascendentes se obtuvo una mediana de 2.00 que correspondiente a un crecimiento lineal, en el caso de las entradas aleatorias, descendientes y los duplicados se obtuvo un resultado de 3.93, 3.85 y 3.93 respectivamentes, respaldando el crecimiento cuadrático

Merge Sort: Se predijo una complejidad (O(n\log n)) para todos los casos. Los resultados respaldan esta predicción, ya que las medianas de las razones fueron 2.13 para entradas aleatorias, 2.11 para ascendentes, 2.12 para descendentes y 2.11 para duplicados. Estos valores, ligeramente superiores a 2, son consistentes con el crecimiento esperado de este algoritmo y muestran que el tipo de entrada no afecta significativamente su rendimiento.

QuickSort: En la Parte I se predijo una complejidad (O(n\log n)) para el mejor y caso promedio, y (O(n^2)) para el peor caso. Los resultados muestran esta diferencia: para entradas aleatorias se obtuvo una mediana de 2.14, cercana al comportamiento

(O(n\log n)), mientras que para entradas ascendentes, descendentes y con duplicados se obtuvieron 3.71, 3.91 y 3.77, respectivamente, valores cercanos al crecimiento cuadrático. Además, se presentaron desbordamientos de pila en tamaños mayores para estas últimas entradas, lo que evidencia las limitaciones de utilizar un pivote fijo.

Radix Sort: En la Parte I se predijo una complejidad (O(n\log n)) para todos los casos. Sin embargo, las mediciones muestran un comportamiento cercano al lineal, con medianas de 2.02 para entradas aleatorias, 2.01 para ascendentes, 2.01 para descendentes y aproximadamente 2.01 para duplicados. Esto se debe a que, al mantener fija la cantidad de dígitos y la base utilizada, el tiempo de ejecución crece aproximadamente de forma proporcional al tamaño del arreglo. La prueba especial también mostró que aumentar la cantidad de dígitos incrementa el tiempo de ejecución, por lo que la predicción inicial no describe correctamente el comportamiento de esta implementación.

TEC | Tecnológico de Costa Rica

Referencias y Declaraciones de Uso de IA:

Ulfran Rodriguez

Prompt 1: Análisis del enunciado

Claude modelo Sonnet 5.5 se utilizó el siguiente prompt, adjuntando el archivo explicativo de la tarea
"*# Proyecto: Tarea Extraclase 2 – Benchmark de algoritmos de ordenamiento*

Actúa como un desarrollador senior especializado en C#, algoritmos y estructuras de datos, y como tutor universitario de Algoritmos y Estructuras de Datos I del Tecnológico de Costa Rica.

Necesito desarrollar individualmente la Tarea Extraclase #2 del curso CE1103, cuyo objetivo es comprobar experimentalmente el comportamiento asintótico de siete algoritmos de ordenamiento.

1. Objetivo general

Desarrollar una aplicación de consola en C# que permita implementar, ejecutar, validar y medir el rendimiento de los siguientes algoritmos:

Selection Sort.

Bubble Sort.

Insertion Sort.

Shell Sort.

Merge Sort.

Quicksort.

TEC | Tecnológico de Costa Rica

Radix Sort.

El propósito no es únicamente ordenar números, sino realizar experimentos que permitan comparar el rendimiento real de cada algoritmo con su complejidad teórica.

2. Requisitos de programación

Utilizar C# y .NET.

Crear un proyecto de consola organizado y modular.

Implementar cada algoritmo de manera independiente.

No utilizar métodos de ordenamiento de bibliotecas para implementar los algoritmos.

Utilizar Array.Sort() únicamente como referencia para validar los resultados.

No agregar optimizaciones que alteren el comportamiento de las versiones solicitadas.

Quicksort debe utilizar obligatoriamente un pivote fijo: primer o último elemento.

Documentar el funcionamiento de los algoritmos mediante comentarios claros.

Quiero que el código sea sencillo de comprender para un estudiante universitario que está aprendiendo programación.

3. Sistema de experimentación

La aplicación debe generar cuatro tipos de arreglos:

Aleatorios.

Ordenados ascendentemente.

Ordenados descendentemente.

Con muchos duplicados, utilizando valores entre 0 y 10.

Los números deben estar entre 0 y 999999.

Tamaños de prueba

Selection Sort, Bubble Sort e Insertion Sort:

1000, 2000, 4000, 8000 y 16000 elementos.

Shell Sort, Merge Sort, Quicksort y Radix Sort:

1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000, 256000, 512000 y 1000000 elementos.

Se debe permitir reducir los tamaños cuando existan limitaciones de hardware o tiempos de ejecución excesivos, dejando constancia de ello.

Reglas de medición

Utilizar System.Diagnostics.Stopwatch.

No medir la generación de los arreglos.

Todos los algoritmos deben recibir exactamente los mismos datos para cada combinación de tamaño y tipo de entrada.

Realizar al menos cinco mediciones por combinación.

Calcular y reportar la mediana.

Descartar la primera ejecución de calentamiento JIT para cada tamaño y algoritmo.

No incluir la copia de los arreglos dentro del tiempo medido.

TEC | Tecnológico de Costa Rica

Ejecutar todos los experimentos en la misma computadora.

4. Validación

Antes de realizar cualquier benchmark, comprobar que cada algoritmo ordena correctamente.

Para ello:

1. Generar un arreglo.

2. Crear dos copias idénticas.

3. Ordenar una con el algoritmo implementado.

4. Ordenar la otra con Array.Sort().

5. Comparar ambos resultados.

6. Registrar si la prueba fue exitosa.

Si un algoritmo falla, detener su experimentación y reportar el error.

5. Prueba especial de Radix Sort

Implementar una prueba adicional para Radix Sort.

Mantener fijo n = 100000 y variar la cantidad de dígitos de los números:

3 dígitos.

6 dígitos.

9 dígitos.

Los números pueden llegar hasta 999999999.

Medir el tiempo de ejecución y comparar cómo influye la cantidad de dígitos en el rendimiento.

6. Archivo CSV

El programa debe guardar automáticamente los resultados en un archivo CSV.

Incluir como mínimo:

Nombre del algoritmo.

Tipo de entrada.

Tamaño del arreglo.

Número de repetición.

Tiempo de ejecución en milisegundos.

Mediana de los tiempos.

Estado de validación.

También debe permitir calcular posteriormente las razones t(2n)/t(n).

No inventar mediciones ni resultados.

7. Análisis de resultados

TEC | Tecnológico de Costa Rica

Preparar los datos necesarios para:

Calcular las razones t(2n)/t(n) para cada algoritmo y tipo de entrada.

Generar una gráfica log-log con todos los algoritmos sobre entradas aleatorias.

Generar una gráfica comparativa de los cuatro tipos de entrada para Insertion Sort.

Generar una gráfica comparativa de los cuatro tipos de entrada para Quicksort.

Comparar los resultados experimentales con las predicciones teóricas.

Las gráficas deben poder generarse a partir del CSV, sin introducir manualmente los datos.

8. Organización del proyecto

Organizar el código de manera modular. Propongo esta estructura:

TareaExtraclase2/

- Program.cs

- Algoritmos/

- SelectionSort.cs

- BubbleSort.cs

- InsertionSort.cs

- ShellSort.cs

- MergeSort.cs

- QuickSort.cs

TEC | Tecnológico de Costa Rica

- *RadixSort.cs*

- *Generadores/*

  - *GeneradorArreglos.cs*

- *Benchmark/*

  - *EjecutorBenchmark.cs*

  - *Medicion.cs*

- *Validacion/*

  - *Validador.cs*

- *Resultados/*

  - *Mediciones.csv*

- *Analisis/*

  - *GeneradorGraficas.cs*

- *README.md*

Puedes proponer cambios si existe una estructura más conveniente.

Soy estudiante de Ingeniería en Computadores del TEC.

Quiero que tengas presente que todavía estoy desarrollando mis habilidades de programación. Necesito código funcional, pero también necesito comprenderlo”

TEC | Tecnológico de Costa Rica

Prompt 2: Algoritmos y array:

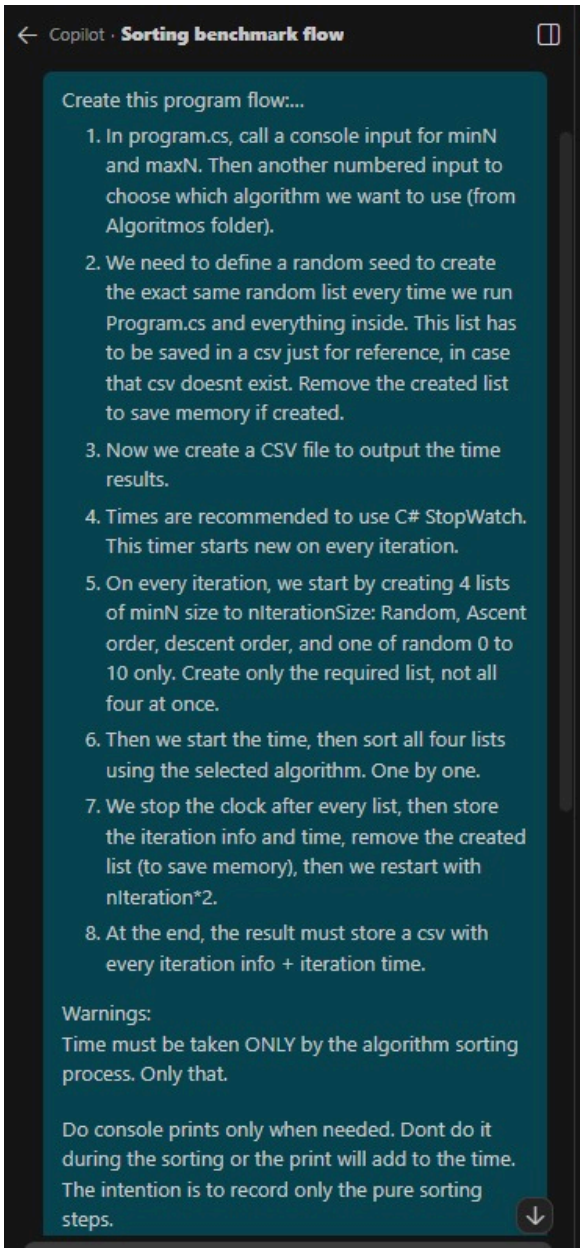
"*Continuando con los algoritmos y con base al contexto anterior, escribe los 7 algoritmos de ordenamiento sin ninguna optimización, en el caso del quicksort tienes que fijar un pivote permanente para que no lo optimice. Al final ocupo que entregues un código para hacer una comprobación rápida de que los algoritmos funcionan, puede ser una validación formal por ejemplo un* ***Array.Sort()***

TEC | Tecnológico de Costa Rica

Ignacio Apuy

Prompt 3: Programa de pruebas:

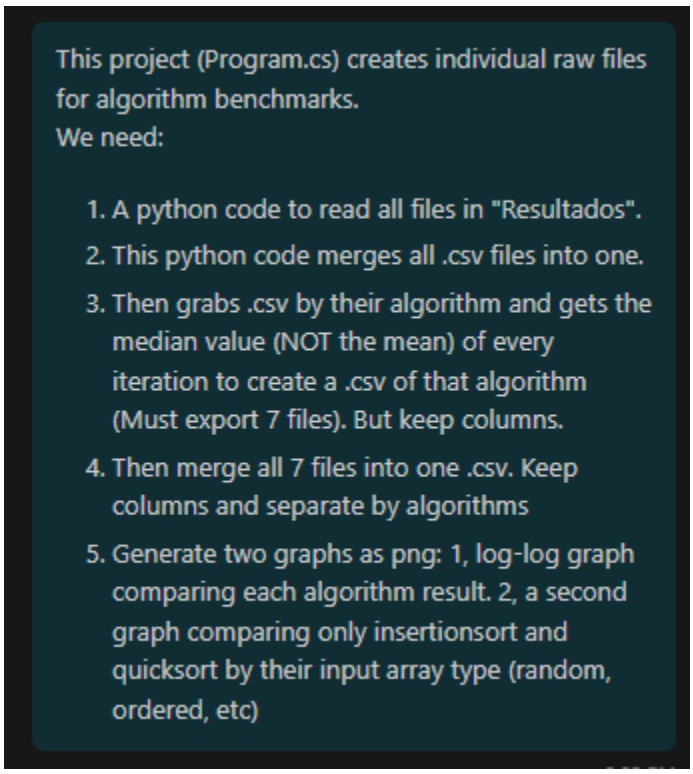
Se le pidió a Copilot un programa con el flujo de pruebas por algoritmo, que requiere de entradas (mínimo, máximo, algoritmo a utilizar), utilizando generadores y contadores, con una semilla por defecto, y finalmente generación de archivos csv.



TEC | Tecnológico de Costa Rica

Prompt 4: Programa graficador

Seguidamente, se le pidió a Copilot crear un archivo en Python que recopilara los archivos csv, los filtrara, obtuviera sus medianas y finalmente realizase gráficas utilizando la librería matplotlib.



TEC | Tecnológico de Costa Rica

Prompt 5 y 6: Arreglos mínimos de errores.

